

Podstawowe dane

Imie i nazwisko	Wojciech Kaźmierczak Robert Mesek
Data	01.01.2024
Tytuł	Projekt z Algorytmów Geometrycznych
Numer projektu	7

Dane techniczne

Procesor	Intel i7-1165G7 x86_64
Język	Python 3.11.5
Biblioteki	numpy 1.25.2, pandas 2.0.3, matplotlib 3.7.2, notebook 6.5.4, sortedcontainers 2.4.0
System operacyjny	WSL Ubuntu 22.04 (MS Windows 10 22H2)
Środowisko	conda 23.11.0

Opis projektu

Należy zaimplementować algorytmy konstrukcji triangulacji dla dowolnego wielokąta prostego dwoma metodami. W obu przypadkach triangulacja ma się opierać jedynie na wierzchołkach wielokąta (bez dodawania wierzchołków wewnątrz wielokąta).

Pierwsza metoda to podział wielokąta na wielokąty monotoniczne (zgodnie z algorytmem podanym na wykładzie), a następnie triangulacja wynikłych z podziału wielokątów monotonicznych także metodą podaną na wykładzie (i realizowaną na ćwiczeniach).

Drugi algorytm to triangulacja Delaunay'a z ograniczeniami (z procedurami odzyskania krawędzi i usunięcia trójkątów zewnętrznych). W tej metodzie wybrać odpowiedni sposób poszukiwania trójkąta w istniejącej triangulacji (i go omówić).

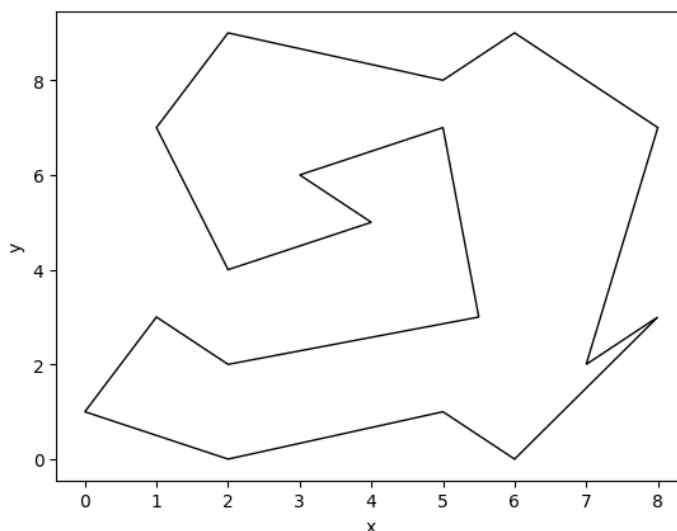
Triangulacja polegająca na podziale wielokątów

Idea algorytmu

Celem algorytmu jest znalezienie triangulacji dowolnego wielokąta prostego. Aby to osiągnąć w tym sposobie musimy przeprowadzić kilka kroków:

1. Klasyfikacja wierzchołków danego wielokąta.
2. Podział wielokąta na wielokąty monotoniczne.
3. Triangulacja wielokątów monotonicznych.

Wielokąt na którym przedstawimy ideę



Rys. 1: Przykładowy wielokąt prosty.

1. Klasyfikacja wierzchołków danego wielokąta

Typu wierzchołków

Wierzchołki naszego wielokąta możemy podzielić na parę kategorii:

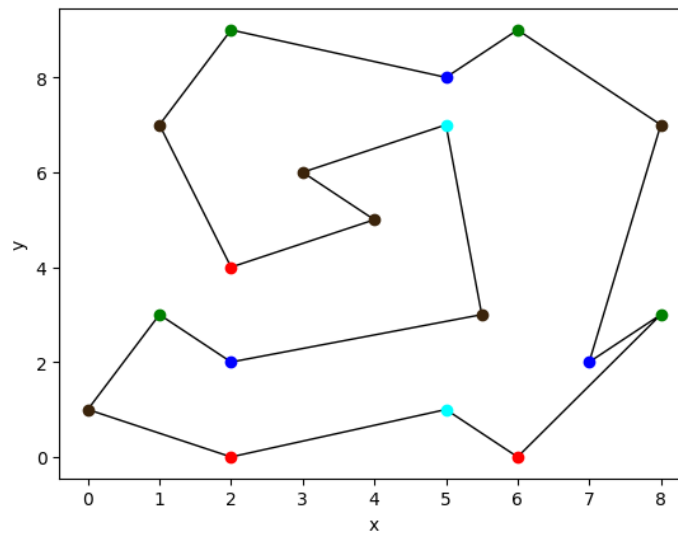
- początkowe, gdy obaj jego sąsiedzi leżą poniżej i kąt wewnętrzny ma mniej niż 180 stopni. To wierzchołki, w których zaczyna się monotoniczny spadek
- końcowe, gdy obaj jego sąsiedzi leżą powyżej i kąt wewnętrzny ma mniej niż 180 stopni. To wierzchołki, w których monotoniczność wielokąta się zmienia, czyli na przykład zaczyna się monotoniczny wzrost, jeśli wcześniej był spadek, lub na odwrót.
- dzielące, gdy obaj jego sąsiedzi leżą poniżej i kąt wewnętrzny ma więcej niż 180 stopni. To wierzchołki, które wyznaczają przekątne (linie łączące), tworzące trójkąty podczas triangulacji.
- łączące, gdy obaj jego sąsiedzi leżą powyżej i kąt wewnętrzny ma więcej niż 180 stopni. To wierzchołki, które są połączone liniami (przekątnymi) wewnątrz wielokąta, tworząc trójkąty.
- prawidłowe, pozostałe przypadki, jeden sąsiad powyżej, drugi poniżej

Wierzchołki łączące i dzielące odgrywają kluczową rolę w procesie triangulacji wielokątów, pozwalając na podział figury na trójkąty w sposób bezkolizyjny.

Oznaczenia wierzchołków kolorami

- Początkowy - kolor zielony
- Końcowy - kolor czerwony
- Łączący - kolor niebieski
- Dzielący - kolor cyjan
- Prawidłowy - kolor brązowy

Rezultat kolorowania



Rys. 2: Przykładowy wielokąt po klasyfikacji.

Wykorzystane funkcje

`classify(a: Point, b: Point, c: Point, eps) -> PointType`

Zwraca typ punktu, wykorzystując własność wyznacznika macierzy liczonego ze wzoru:

`det = (a.x - c.x) * (b.y - c.y) - (a.y - c.y) * (b.x - c.x)`

2. Podział wielokąta na wielokąty monotoniczne

Konwersja wielokąta na podwójnie łączoną listę krawędzi

Dla naszego algorytmu pomocne jest sporządzenie struktury, w której będziemy mieli możliwość szybkiego dostępu do elementów wielokąta takich jak: kolejne krawędzie wraz z punktem będącym jej pomocnikiem, punkty z informacją do jakiej krawędzi należą.

Rozwiązanie będzie korzystać z algorytmu zmiatania

W tym celu będziemy potrzebować 2 pomocniczych struktur danych:

- Q - struktura zdarzeń - kolejka priorytetowa - są to nasze punkty, gdzie największy priorytet ma górny lewy punkt
- T - struktura stanu - dynamiczne drzewo wyszukiwań binarnych - są to krawędzie posortowane od lewej strony do prawej w miejscu przecięć z miotłą

Przebieg algorytmu

Kolejne punkty z kolejki priorytetowej Q będziemy przetwarzać w sposób zależny od typu danego punktu. Głównym celem jest wyeliminowanie wierzchołków łączących oraz dzielących. Wielokąt y-monotoniczny to takich, który właśnie nie posiada żadnych wierzchołków tego typu.

W tym celu dla każdego wierzchołka dzielącego dodajemy przekątne wychodzące w górę, a dla każdego wierzchołka łączącego wychodzące w dół. Przekątne te mają nie przecinać żadnej innej przekątnej.

Będziemy starali się znaleźć wierzchołki znajdujące się w pobliżu rozpatrywanego aktualnie wierzchołka v_i . Aby szybko znajdować taki wierzchołek dla v_i dzielącego zauważmy, że będzie on najniższym wierzchołkiem leżącym pomiędzy prostą e_l będącą bezpośrednio na lewo od v_i oraz e_p będącą bezpośrednio po prawej stronie. Wierzchołek ten nazwiemy pomocnikiem(v_i), zauważmy, że może on być górnym końcem samej e_l .

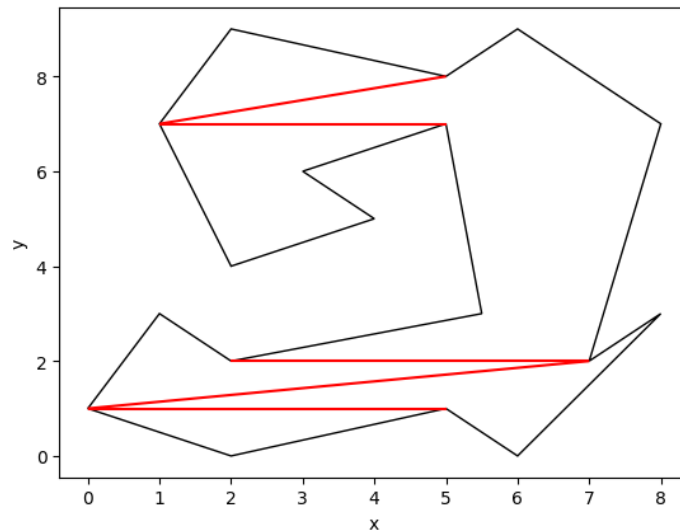
Pozostają jeszcze wierzchołki v_i łączące. Potrzebować one będą wierzchołków znajdujących się pod miotłą (jeszcze nie przetworzonych), ale będzie to dokładnie odwrócona sytuacja dla wierzchołka dzielącego. Za pomocnika(v_i), będziemy wybierać wierzchołek leżący pomiędzy e_l a e_p , znajdujący się najbliżej.

Dowód poprawności oraz wyprowadzenie

Dowód poprawności oraz szczegóły wyprowadzenia algorytmu znajdziemy w książce

[1] "Geometria obliczeniowa. Algorytmy i zastosowania" M. Berg

Rezultat podziału



Rys. 3: Przykładowy wielokąt po podziale.

Wykorzystane funkcje

`make_monotone(polygon) -> tuple[list[Edge], list[Edge]]`

Główna funkcja wykonująca algorytm zmiatania. Przyjmuje ona wielokąt w postaci listy kolejnych punktów w kierunku przeciwnym do ruchu wskazówek zegara. Zwraca ona listę krawędzi wielokąta oraz listę nowo dodanych wierzchołków.

`handle_start_vertex(T: SortedList, color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki początkowe.

`handle_end_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki końcowe.

`handle_split_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki dzielące.

`handle_merge_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki łączące.

`handle_regular_vertex(T: SortedList, D: list[Edge], color_point: ColorPoint)`

Funkcja przetwarzająca wierzchołki prawidłowe.

`get_points_and_diagonals(polygon: list[tuple[float, float]])`

Pomocnicza funkcja przetwarzająca wynik działania `make_monotone()` na listę współrzędnych punktów oraz krawędzi w postaci indeksów, które w punkty należy połączyć w liście współrzędnych.

3. Triangulacja wielokątów monotonicznych

Konwersja punktów i przekątnych na poszczególne wielokąty y-monotoniczne

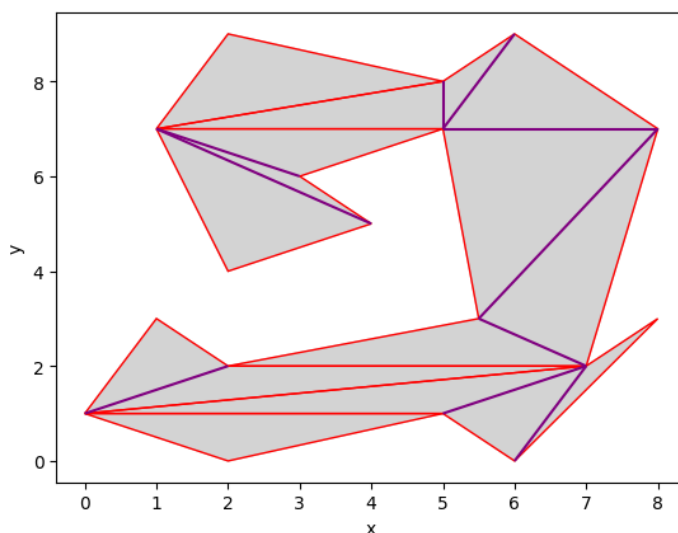
Sporządzimy listę punktów, jako strukturę gdzie każdy punkt posiada zbiór innych punktów z których możemy do niego dotrzeć na podstawie otrzymanych przekątnych.

Posłużymy się następnie lekko zmodyfikowanym algorytmem przeszukiwania wzdłuż. Dla każdego punktu będziemy szukać ścieżek w których wracamy do punktu startu. Jedyne na co musimy uważać, to aby od razu nie wrócić się do tego samego punktu. Po znalezieniu takiej ścieżki usuwamy połączenia ze struktury i próbujemy dla kolejnego punktu.

Triangulacja kolejnych wielokątów y-monotonicznych

- Określamy lewy i prawy łańcuch wielokąta względem kierunku monotoniczności
- Porządkujemy wierzchołki wzdłuż kierunku monotoniczności
- Wkładamy dwa pierwsze wierzchołki na stos.
- Jeśli kolejny wierzchołek należy do innego łańcucha niż wierzchołek stanowiący szczyt stosu, to możemy go połączyć ze wszystkimi wierzchołkami na stosie. Na stosie zostają dwa wierzchołki, które były „zamiatane” ostatnie.
- Jeśli kolejny wierzchołek należy do tego samego łańcucha co wierzchołek ze szczytu stosu, to analizujemy kolejne trójkąty, jakie tworzy dany wierzchołek z wierzchołkami zdejmowanymi ze stosu:
 - Jeśli trójkąt należy do wielokąta, to dodajemy przekątną i usuwamy odpowiedni wierzchołek ze szczytu stosu
 - w przeciwnym przypadku umieszczamy badane wierzchołki na stosie.

Rezultat triangulacji



Rys. 4: Przykładowy wielokąt po triangulacji.

Wykorzystane funkcje

`triangulate_y_monotone(polygon: list[tuple[float, float]])`

Główna funkcja, która dla znalezionej wielokąta y-monotonicznego znajdzie zwróci jego przekątne, które należy dodać w postaci współrzędnych końców przekątnych.

`find_chains(points: list[Point]) -> tuple[set[int], set[int]]`

Funkcja tworząca łańcuch lewy i prawy.

`triangle_in_polygon(chain: set[int], a: Point, b: Point, c: Point)`

Funkcja sprawdzająca czy trójkąt znajduje się wewnątrz wielokąta, korzysta z wyznacznika macierzy.

`check_same_chains(left_chain: set[int], right_chain: set[int], p1: Point, p2: Point)`

Funkcja sprawdzająca czy dwa punkty znajdują się na tych samych czy różnych łańcuchach.

`check_neighbours(points: list[Point], a: Point, b: Point)`

Funkcja sprawdzająca czy punkty są dla siebie sąsiadami.

Kompletny algorytm

Dane wejściowe

Wielokąt prosty zadany w postaci punktów w kierunku przeciwnym do wskazówek zegara.

Rezultat

Triangulacja wielokąta prostego w postaci przekątnych, które należy dołożyć.

Złożoność czasowa

Podział wielokąta o n wierzchołkach na wielokąty y-monotoniczne: czas $O(n \cdot \log(n))$, pamięć $O(n)$

Triangulacja y-monotonicznego wielokąta o n wierzchołkach $O(n)$

Złożoność czasowa kompletnego algorytmu dla n wierzchołków: $O(n \cdot \log(n))$

Złożoność pamięciowa kompletnego algorytmu dla n wierzchołków: $O(n)$

Bibliografia

- [1] "Geometria obliczeniowa. Algorytmy i zastosowania" M. Berg M. Kreveld M. Overmars
ISBN: 9788320432442 WNT Warszawa 2007

Triangulacja Delaunay'a z ograniczeniami

Porównanie algorytmów